



GUIA BÁSICA DE ARQUITECTURA DE MICROSERVICIOS

Roberto Carlos Catuta

Guía Básica con Herramientas Clave y Diagramas

INDICE

1. Introducción

1.1 ¿Qué es la Arquitectura de Microservicios?

1.2 Ejemplo: Monolítico vs. Microservicios

2. Conceptos Básicos

2.1 Componentes Clave

2.2 Comunicación y Orquestación

3. Herramientas Clave

3.1 Desarrollo y Despliegue

3.2 Comunicación

3.3 Seguridad

4. Diagramas Básicos

4.1 Arquitectura General

4.2 Comunicación con Kafka

5. Patrones Comunes

5.1 Circuit Breaker

5.2 SAGA

5.3 CQRS

6. Ejemplo Práctico: Tienda Online

6.1 Pasos para Implementar

7. Desafíos y Soluciones

8. Conclusión

9. Bibliografía

1. INTRODUCCIÓN

¿Qué es la Arquitectura de Microservicios?

Los microservicios son un estilo de arquitectura donde una aplicación se divide en **servicios pequeños, independientes y especializados**, que se comunican entre sí. A diferencia de las aplicaciones monolíticas (todo en un solo bloque), los microservicios permiten:

- **Escalabilidad independiente** (cada servicio puede crecer según su demanda).
- **Mayor resiliencia** (si un servicio falla, no colapsa todo el sistema).
- **Libertad tecnológica** (cada servicio puede usar diferentes lenguajes/frameworks).

Ejemplo:

- **Monolítico:** Una tienda online donde el carrito, pagos y usuarios están en un solo código.
- **Microservicios:**
 - **Servicio de Usuarios** (Node.js + MongoDB).
 - **Servicio de Pagos** (Java + PostgreSQL).
 - **Servicio de Catálogo** (Python + Redis).

2. CONCEPTOS BÁSICOS

A. Componentes Clave

1. **Servicios Independientes:** Cada uno tiene su propia base de datos y lógica.
2. **API Gateway:** La "puerta de entrada" que dirige las peticiones a los microservicios.
3. **Comunicación:** HTTP/REST, gRPC o mensajería (Kafka, RabbitMQ).
4. **Orquestación:** Kubernetes o Docker Swarm para gestionar contenedores.

B. Ventajas

Microservicios	Monolítico
Escalabilidad granular	Escalabilidad limitada
Fallos aislados	Fallo = Todo caído
Tecnologías diversas	Todo en un solo stack

3. HERRAMIENTAS CLAVE

A. Desarrollo y Despliegue

- **Docker:** Empaqueta cada servicio en un contenedor.
- **Kubernetes (K8s):** Orquesta contenedores automáticamente.
- **Azure AKS / AWS EKS:** Servicios gestionados de Kubernetes en la nube.

B. Comunicación

- **HTTP/REST:** Para solicitudes simples (ej: GET /users).
- **gRPC:** Para alta velocidad (ej: transacciones bancarias).
- **Apache Kafka:** Para eventos asíncronos (ej: notificaciones).

C. Seguridad

- **OAuth2.0 / JWT:** Autenticación entre servicios.
- **Azure Key Vault / HashiCorp Vault:** Guarda contraseñas y certificados.

4. DIAGRAMAS BÁSICOS

Diagrama 1: Arquitectura General

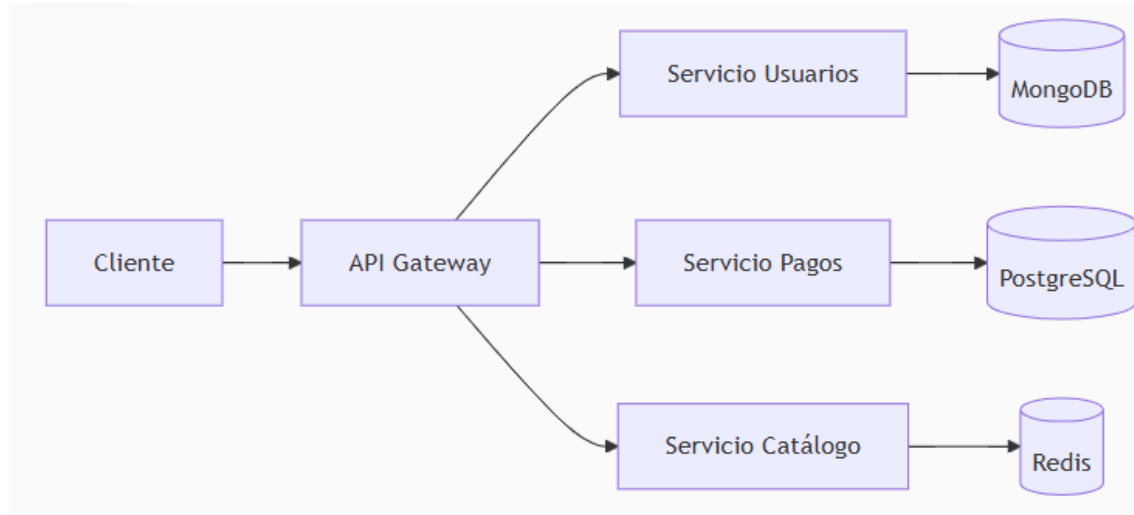
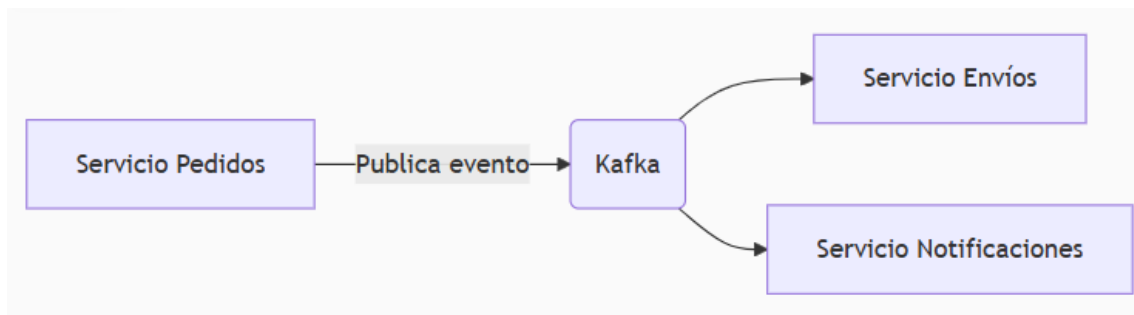


Diagrama 2: Comunicación con Kafka



5. PATRONES COMUNES

A. Circuit Breaker

Evita que un servicio colapse al fallar. **Herramienta:** Polly (en .NET), Resilience4j (Java).

B. SAGA

Coordina transacciones distribuidas. **Ejemplo:**

1. Pedido creado → Servicio A.

- 2. Si Servicio B falla, Servicio A compensa (ej: cancela el pedido).

C. CQRS

Separa consultas (lecturas) de comandos (escrituras). **Herramienta:** Azure Cosmos DB.

6. EJEMPLO PRÁCTICO: TIENDA ONLINE

Pasos para Implementar

- 1. **Dividir la aplicación:**
 - Usuarios, Productos, Pedidos, Pagos.
- 2. **Desplegar en Kubernetes:**
kubectI create deployment servicio-usuarios --image=node-usuarios
- 3 **Configurar API Gateway:**
 - Usar **Azure API Management** o **Kong**.

7. DESAFÍOS Y SOLUCIONES

Desafío	Solución
Latencia en llamadas	Usar gRPC o caché (Redis)
Seguridad	JWT + mTLS
Monitoreo	Prometheus + Grafana

8. CONCLUSIÓN

Los microservicios son ideales para aplicaciones complejas y escalables, pero requieren **buena planificación**. Empieza con:

1. **Herramientas sencillas:** Docker + Kubernetes.
2. **Patrones básicos:** Circuit Breaker, SAGA.
3. **Monitoreo constante:** Grafana para métricas.

9. BIBLIOGRAFÍA

Building Microservices" (2ª edición) – Sam Newman (O'Reilly, 2021).

<https://www.oreilly.com/library/view/building-microservices-2nd/9781492034018/>

Microsoft Azure – Arquitectura de Microservicios, <https://learn.microsoft.com/es-es/azure/architecture/guide/architecture-styles/microservices>